HW 1 — Language Models from Scratch (Part 1)

RNN vs. GRU vs. LSTM vs. Transformer on Tiny Shakespeare, with nanolm

Giovanni Cascone · MS Computer Engineering, San José State University · October 29, 2026

Configs, scripts, summary.csv and figures: [insert GitHub / Google Drive link]

1. Question and hypothesis

This assignment asks what the standard language-model metrics mean, and how recurrent networks and Transformers differ in practice when the data, the token budget and the metric are held fixed. Before running anything I expected:

- **Metrics.** Loss, perplexity, bits/token and bits-per-byte (bpb) are one quantity in different units, so the logged values should be reproducible exactly by hand. A model that knows nothing should score about ln V, because it spreads probability evenly over the V tokens.
- **Architectures.** Gates (GRU, LSTM) should beat a vanilla RNN by a wide margin, because without them gradients vanish over long spans. The modern Transformer (RMSNorm + RoPE + SwiGLU) should be best. A larger LSTM might not help: Tiny Shakespeare is only about 1 MB, and 16M training tokens mean about 16 passes over it, so a bigger model can memorize.
- **Context.** Attention can read any earlier position directly, so the Transformer should keep improving with longer windows. The LSTM compresses the past into a fixed-size state and should saturate earlier.
- **Attention.** A small character model should learn simple, readable heads (previous character, start of the current word, line structure) rather than abstract ones.

2. Setup

All runs use the course code `nanolm` without modification. Each experiment is a sweep YAML, and every number below is read from a run directory (`metrics.json`, `eval_log.jsonl`). The one exception is the step-0 check in Table 2, which used a short script (`init_loss.py`) on the same configs.

Table 1. Experimental setup.

item	value
data	Tiny Shakespeare, 1,115,394 characters; last 10% held out for validation (111,360 scored tokens)
tokenizer	characters (V = 65), 1 token = 1 byte; Part 1b also uses byte (256), SentencePiece BPE (512) and word (2048)
token budget	2,000 steps × 8,192 tokens = 16.4M training tokens per run (about 16 passes), the same for every model
models	RNN / GRU / LSTM 2 × 256 (and LSTM 2 × 512); Transformer 4 layers × 256 wide × 4 heads, GPT-2 style (LayerNorm, learned positions, GELU) and modern (RMSNorm, RoPE, SwiGLU)
optimization	AdamW, cosine schedule, 100 warm-up steps; lr 1e-3 / weight decay 0.1 (Transformers), lr 2e-3 / no weight decay (recurrent); dropout 0.1; gradient clipping at 1.0; batch 32 × 256
context study	train.seq_len ∈ {64, 128, 256, 512} with batch × seq_len = 8,192 fixed; evaluation in non-overlapping windows of seq_len
seeds	1337 and 42, the same pair for every configuration (paired t-test from analyze.py)
compute	32 runs, 393M training tokens, about 5.1 × 10^15 FLOPs (6ND)
device	MacBook Pro, Apple M5 Max, 48 GB; PyTorch 2.15.0 on MPS, fp32; macOS 27.0.1, Python 3.14.5
code version	DeepDataMiningLearning commit a4d756e8 (clean, not dirty)
wall-clock	19.5 min for all 32 runs, run concurrently (6 Transformer + 4 recurrent at a time). Per-run train_time_sec (sum 169.7 min) is inflated by sharing the GPU.

3. Results

3.1 Metrics by hand

For the trained modern Transformer (seed 1337), `metrics.json` gives val_loss = 1.467075 nats/token over N_tok = 111,360 tokens covering N_bytes = 111,360 bytes. From these three numbers alone:

Table 2a. Recomputed metrics vs. logged values (gpt_modern, seed 1337).

quantity	formula	by hand	logged	|diff|
loss	mean −ln p (given)	1.467075	1.467075	0
perplexity	e^loss	4.336532	4.336532 (val_ppl)	0
bits / token	loss / ln 2	2.116542	—	—
bytes / token	N_bytes / N_tok	1.000000	1.000000	0
bits per byte	(loss / ln 2) × (N_tok / N_bytes)	2.116542	2.116542 (val_bpb)	< 1e-15

Because every character of this ASCII text is one byte, bits/token equals bpb here. A perplexity of 4.34 means the model is, on average, as unsure as a fair choice among about 4 characters out of 65.

Table 2b. Loss of an untrained Transformer vs. ln V, mean of 2 seeds. *Step 0* = fresh initialization, no update (script). *1 step* = `train.max_steps=1`, the sweep run.

tokenizer	V	ln V	step-0 loss	step-0 / ln V	1-step loss	1-step / ln V	V used in val
char	65	4.174	4.224	1.012	4.168	0.998	61
byte	256	5.545	5.511	0.994	5.442	0.981	61
BPE 512	512	6.238	6.246	1.001	6.227	0.998	248
word 2048	2048	7.625	7.646	1.003	7.568	0.993	1294

All four fresh models start within 1.2% of ln V, as they should: about 4.2 nats for characters (the tutorial reports 4.22), up to 7.6 for words. A model far from ln V at step 0 would have pointed to an initialization or data bug. The single optimizer step of `max_steps=1` then lowers every loss by 0.02–0.08 nats. The drop is largest for byte (0.07) and word (0.08), whose vocabularies contain many entries that never occur in the text: only 61 of 256 byte values and 1,294 of 2,048 words appear in the validation split. Adam's first update moves every weight by about the learning rate regardless of gradient size, so even at warm-up LR it starts pushing those unused tokens down.

3.2 RNN vs. GRU vs. LSTM vs. Transformer

Table 3. Final validation bpb over the full validation split, mean ± std over 2 seeds, with the paired difference vs. the GPT-2-style Transformer. Seed noise floor (median std) = 0.003 bpb.

model	params	val bpb	Δ vs GPT-2 style	paired p	best periodic bpb (at M tokens)
Transformer, modern (4 × 256)	3.16M	2.116 ± 0.001	−0.069	0.004	2.104–2.113 (15.6)
LSTM 2 × 256	1.07M	2.137 ± 0.005	−0.048	0.059	2.134–2.140 (16.4)
GRU 2 × 256	0.81M	2.160 ± 0.002	−0.026	0.005	2.158–2.161 (16.4)
LSTM 2 × 512	4.24M	2.172 ± 0.009	−0.013	0.238	2.121–2.126 (9.0–9.8)
Transformer, GPT-2 style (4 × 256)	3.44M	2.185 ± 0.002	0	—	2.184–2.185 (15.6–16.4)
vanilla RNN 2 × 256	0.28M	2.437 ± 0.007	+0.251	0.010	2.431–2.442 (16.4)

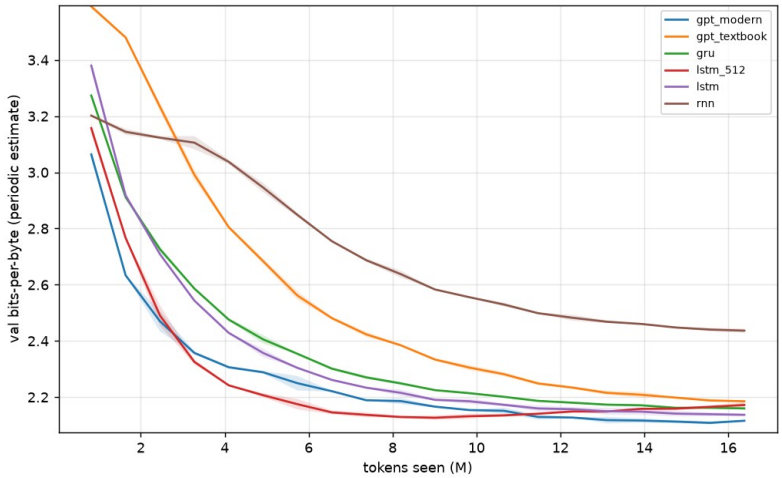


Figure 1. Validation bpb (periodic estimate on 131k tokens) vs. training tokens, mean over 2 seeds (shaded band: ±1 std).

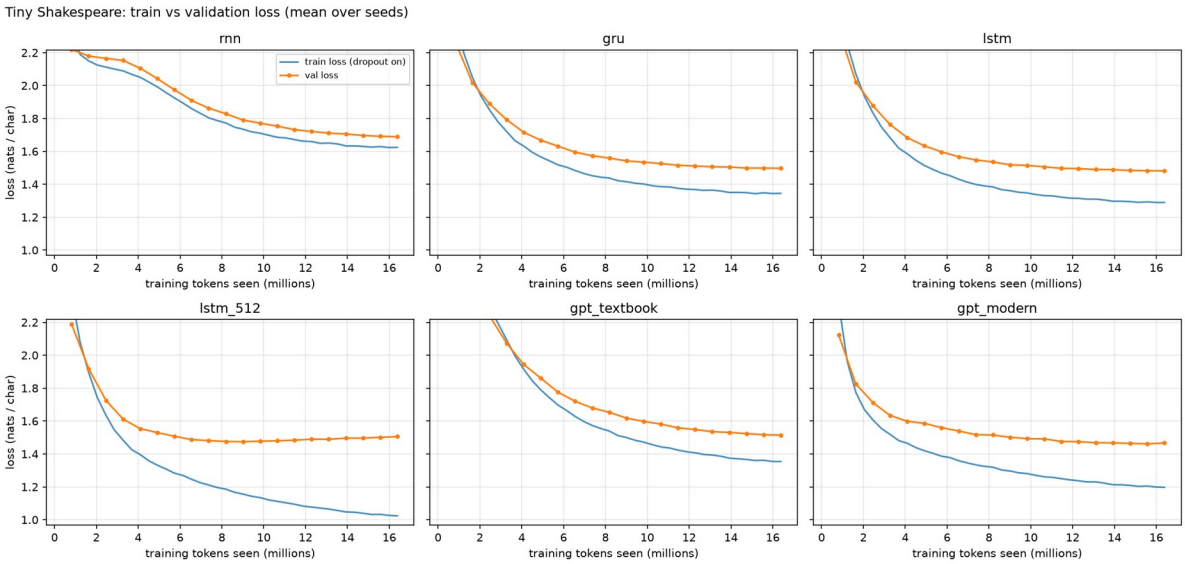


Figure 2. Train loss (dropout on) vs. validation loss in nats/char, per model. Only the 512-wide LSTM's validation loss clearly turns upward.

The ranking matches the tutorial's reference runs to within 0.01 bpb for every model. The differences between the six models (0.013–0.32 bpb) are 4 to 100 times the seed noise floor. Two individual paired p-values are not small (LSTM vs. GPT-2 style p = 0.06; LSTM-512 p = 0.24), because a paired test on n = 2 has a single degree of freedom. For LSTM-512, its seed spread (± 0.009) is as large as the gap it is tested on, so I call it *not resolved* against the GPT-2-style model.

3.3 Context length

Table 4. Final validation bpb vs. train.seq_len (also the evaluation window), mean ± std over 2 seeds. A gain is *resolved* if it exceeds 2 × the noise floor (0.0076).

seq_len	LSTM 2 × 256	gain	Transformer (modern)	gain
64	2.181 ± 0.005	—	2.156 ± 0.001	—
128	2.149 ± 0.003	+0.032 (resolved)	2.114 ± 0.009	+0.043 (resolved)
256	2.137 ± 0.005	+0.012 (resolved)	2.116 ± 0.001	−0.002 (not resolved)
512	2.130 ± 0.006	+0.007 (not resolved)	2.103 ± 0.001	+0.013 (resolved)
total 64 → 512		**0.051**		**0.053**

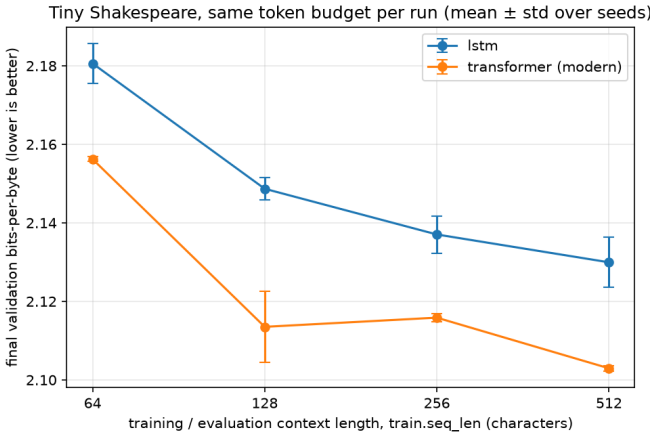


Figure 3. Final validation bpb against context length (log scale), mean ± std over 2 seeds. Every point has the same 16.4M-token budget. Learning curves per context length are in Appendix C.

3.4 Attention analysis

I ran nanolm's attention visualization (`viz attention`, through `attention_heads.py`) on the trained modern Transformer (seed 1337), using three kinds of text. The first is Shakespeare dialogue, which is in-distribution: `ROMEO:⏎But`

`soft! what light through yonder window breaks?⏎JULIET:⏎Ay me!`. The second is modern prose, which is out of distribution. The third is a random 18-character string repeated three times, a copying test. For every head I averaged the attention weight that query position t puts on a specific earlier position, as in §1.6.2. Positions where that target coincides with t or t−1 are excluded. *Chance* is the score of a head that spreads its weight evenly over the visible prefix, the mean of 1/(t+1).

Table 5. Mean attention weight on the relevant position for the three clearest heads (× chance in parentheses).

head type	head	target position	Shakespeare	modern prose	repeated string
previous-token	L2·H2	t − 1	0.49 (9×)	0.57 (11×)	0.64 (10×)
word-start	L2·H0	first letter of current word	0.27 (5×)	0.26 (5×)	0.34 (5×)
line / speaker marker	L3·H2	last ':' or newline	0.37 (11×)	— (no ':' or ↵)	— (no ':' or ↵)
induction (copy)	best: L3·H2	token after previous copy of x_t	—	—	0.04 (1.2×)

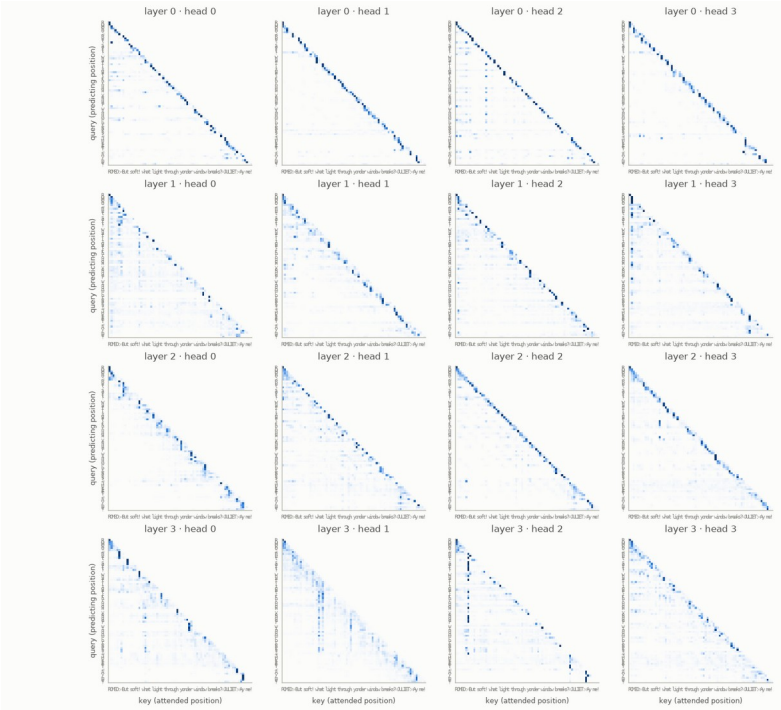


Figure 4. Attention maps on the Shakespeare text (row = query position t, column = attended position; causal mask leaves the upper triangle empty). L2·H2 is a sharp sub-diagonal (previous character). L3·H2 shows vertical stripes at the ':' and newline after ROMEO, which later positions keep attending to. Maps for the other two texts are in Appendix D.

Beyond the three heads in Table 5, all four layer-0 heads lean toward the previous character on the two natural texts (0.31–0.56), acting like learned bigram/trigram features. L3·H2 also has the longest mean look-back of all 16 heads (13.4 characters on both natural texts, against a median of 6–8 for the other heads). This matches the tutorial's description of a head that remembers that a line of dialogue has started.

4. Analysis

Why does the vanilla RNN fall so far behind? It is 0.25 bpb behind the GPT-2-style Transformer and 0.32 behind the best model, about 75–100 times the seed noise. Two pieces of evidence say this is a limit of the architecture, not under-training or a bug. First, it is not overfitting: its train–validation gap is the smallest of all models (0.06 nats, against 0.15–0.27 for the others), so it fails to fit the training data rather than memorizing it. Second, its curve in Figure 1 has a long plateau near 3.1 bpb over the first 3M tokens before it starts improving, the signature of a hard optimization problem. In a plain RNN, $h_t = \tanh(W h_{t-1} + U x_t)$, so the gradient through k steps is a product of k Jacobians. It shrinks or explodes geometrically. Clipping handles the explosion, but nothing fixes the vanishing, so the network effectively learns only short-range statistics. The GRU and LSTM add an additive state path ($c_t = f \odot c_{t-1} + i \odot g$) controlled by gates,

which lets gradients survive many steps. That one change recovers 0.28–0.30 bpb with only 3–4 times more parameters. Being small (0.28M parameters) does not explain the gap: the GRU has 0.81M and is 0.28 bpb better.

Is the larger LSTM better or worse, and what in the curves shows why? It is worse at the end, but better at its best point. Its final bpb is 2.172, against 2.137 for the 2 × 256 LSTM (+0.035, about 10 times the noise floor, and worse in both seeds). But its *best* periodic bpb, 2.121–2.126 at 9–10M tokens, is better than anything the small LSTM or the GRU ever reaches. Figure 2 shows why: after about 9M tokens its validation loss turns upward while its train loss keeps falling to 1.03 nats. That leaves a train–validation gap of 0.48 nats, against 0.19 for the small LSTM. With 4.2M parameters, 1 MB of text and about 16 passes, the larger LSTM memorizes the training set. This is a genuine negative result (overfitting), not a bug: both seeds show it, the training curve is smooth and keeps improving, and the best checkpoint is excellent. Early stopping at about 9M tokens, or more dropout or weight decay, would fix it. The parameter count alone is not the explanation either: the 3.2M-parameter Transformer sees the same data but generalizes better (gap 0.27).

Which architecture uses longer context more effectively? The Transformer, though the effect at this scale is smaller than I expected. It is better at every context length, and it is the only model that still gains a resolved amount from 256 to 512 (+0.013, while the LSTM's +0.007 is within noise). The LSTM's gains shrink steadily with each doubling (0.032, 0.012, 0.007) and stop being measurable after 256 characters. The Transformer's two gains (0.043 at 128 and 0.013 at 512) are separated by a flat step from 128 to 256. I treat that flat step as noise rather than a bug: the 128 point has an unusually large seed spread (± 0.009, versus ± 0.001 for the other Transformer points), and the learning curves for 128, 256 and 512 lie on top of each other (Appendix C). Over the whole range from 64 to 512, both models improve by about the same total (0.051 vs. 0.053), so the honest claim is *"no stopping point visible for the Transformer up to 512; the LSTM stops benefiting at about 256"*, not *"far better use of context"*. Two caveats. First, because evaluation uses non-overlapping windows, a character is predicted from on average only about seq_len/2 earlier characters. Second, the LSTM is reset to a zero state at every window, so part of its gain from longer windows comes simply from fewer cold starts, not from using distant information.

Attention heads. The previous-token and word-start heads are stable across all three texts: the same heads, at similar weights (5–11 × chance). So they are real mechanisms, not artifacts of one sentence. The line-marker head only has a target in the Shakespeare text, where it scores 11 × chance. No head performs induction (copying from an earlier repetition): the best score is 1.2 × chance. I read this as a genuine negative result rather than an analysis bug, since the same metric code finds the other patterns at 5–11 × chance. A 4-layer, 3M-parameter model trained on 1 MB of text without exact repetitions has little reason to learn to copy. L3·H1's block pattern on the repeated string (Appendix D) attends broadly to the earlier copy, but not to the specific next character.

5. Conclusion and limitations

Loss, perplexity and bpb are the same quantity in different units, and the logged values are reproduced exactly from val_loss, val_tokens and val_bytes. Fresh models start within about 1% of ln V for all four tokenizers. With the budget held at 16.4M characters, gating closes almost all of the RNN's gap. The modern Transformer is the best model (2.116 bpb), and its recipe matters as much as the architecture: the GPT-2-style Transformer is worse than a small LSTM. A larger LSTM overfits this 1 MB corpus, so curves rather than final numbers are needed to judge it. The Transformer keeps benefiting from context up to 512 characters, while the LSTM levels off at about 256. Its attention heads are easy to read: previous character, word start, and line structure.

Limitations. Only 2 seeds: the noise floor (0.003 bpb) is itself an estimate from 2 samples, and paired p-values have one degree of freedom. One dataset, one model size per family, and the learning rates were taken from the course configs rather than tuned per model, so the RNN's gap may be partly a tuning gap. The context study changes the training and evaluation context together. A sliding-window evaluation at a fixed long context would separate the two. The attention numbers come from three short texts (54–78 characters) and one seed. All runs used Apple MPS in fp32, and per-run timings are not comparable because the runs shared the GPU.

Appendix

A. Untrained models: per-seed values

Table A1. Step-0 (fresh initialization) and 1-step losses by seed. bpb of a uniform guess = log2 V × tokens/byte.

tokenizer	seed	ln V	step-0 loss	1-step loss	1-step bpb	uniform bpb	bytes/token
char	1337	4.1744	4.2164	4.1601	6.002	6.022	1.000
char	42	4.1744	4.2319	4.1758	6.024	6.022	1.000
byte	1337	5.5452	5.5119	5.4436	7.853	8.000	1.000
byte	42	5.5452	5.5091	5.4404	7.849	8.000	1.000
BPE 512	1337	6.2383	6.2525	6.2347	5.117	5.120	1.758
BPE 512	42	6.2383	6.2385	6.2190	5.104	5.120	1.758
word 2048	1337	7.6246	7.6796	7.6017	6.760	6.781	1.622
word 2048	42	7.6246	7.6116	7.5338	6.700	6.781	1.622

Per token, the losses rise with V (4.2 → 7.6 nats). Per byte, the untrained BPE model is the *least* bad (5.1 bpb), because each of its tokens covers 1.76 bytes. This is the reason only bpb can compare tokenizers fairly (§1.2.3).

B. Best vs. final checkpoint and train–val gap

Table B1. Periodic validation estimate, best and final, and the train–validation loss gap at the end of training (mean of 2 seeds).

model	best bpb	at M tokens	final bpb	final − best	train loss	val loss	gap
rnn	2.437	16.4	2.437	0.000	1.626	1.689	+0.063
gru	2.160	16.4	2.160	0.000	1.346	1.497	+0.151
lstm	2.137	16.4	2.137	0.000	1.291	1.482	+0.191
lstm_512	2.123	9.0–9.8	2.172	+0.049	1.028	1.506	+0.478
gpt_textbook	2.185	15.6–16.4	2.185	+0.001	1.357	1.515	+0.158
gpt_modern	2.108	15.6	2.116	+0.008	1.201	1.467	+0.266

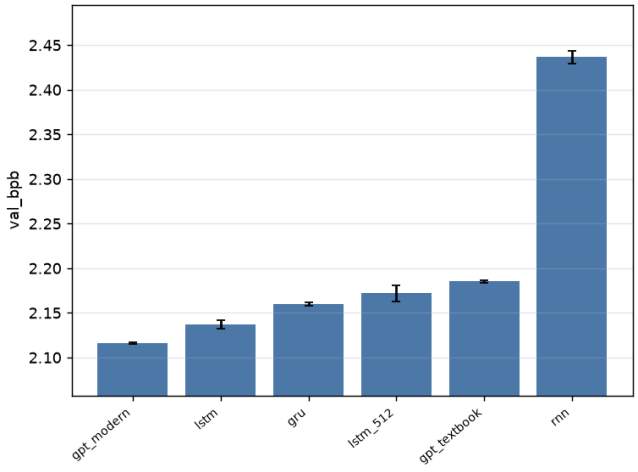


Figure B1. Final validation bpb (bits per byte) per model, error bars = ±1 std over 2 seeds (analyze.py bars.png).

C. Context-length learning curves

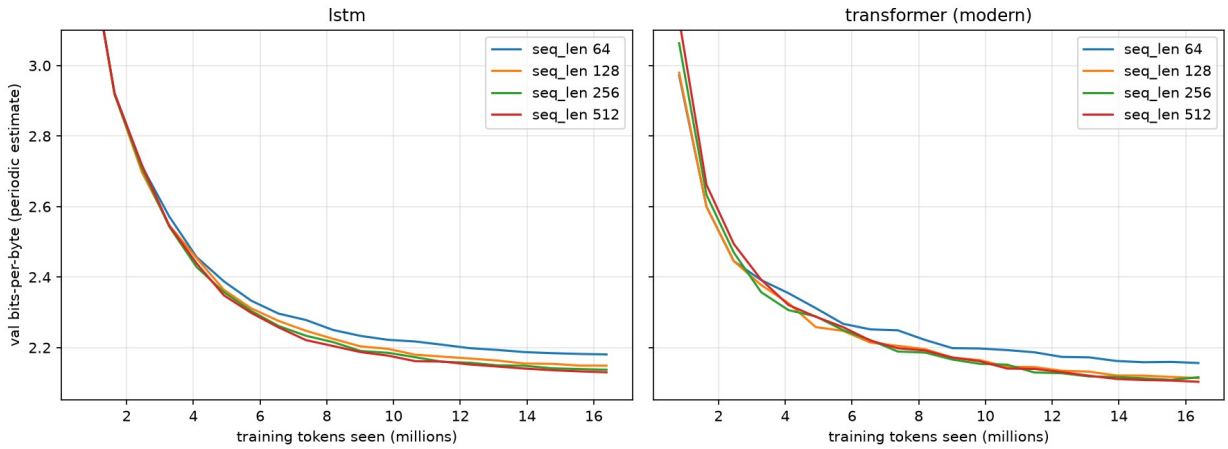


Figure C1. Validation bpb vs. training tokens for each context length (mean of 2 seeds). For the LSTM, the 64-character curve separates after about 4M tokens. For the Transformer, the 128/256/512 curves overlap until the end of training.

D. Attention maps for the other two texts

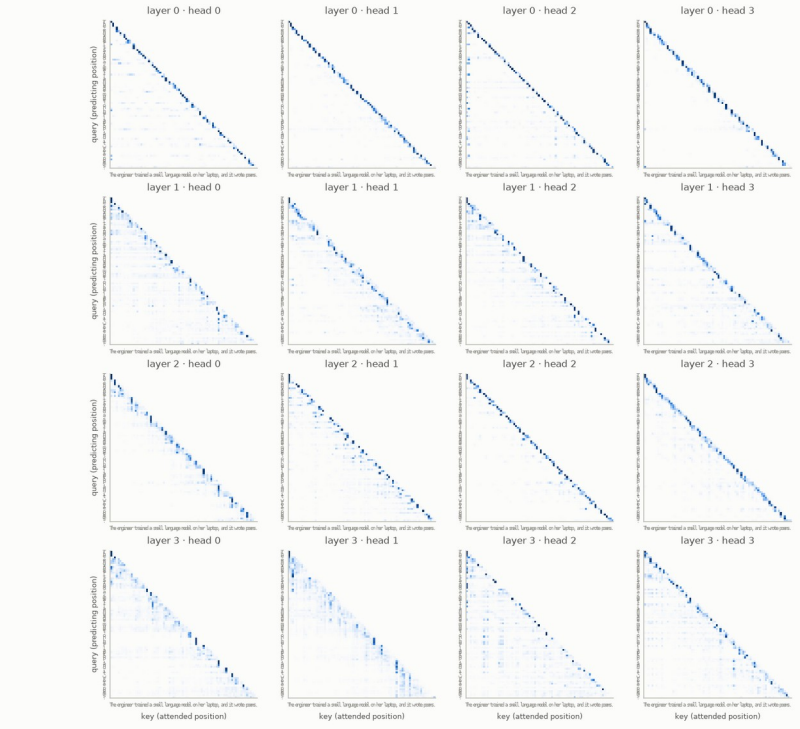


Figure D1. Modern prose: `The engineer trained a small language model on her laptop, and it wrote poems.` The same previous-token (L2·H2) and word-start (L2·H0) heads appear as on Shakespeare.

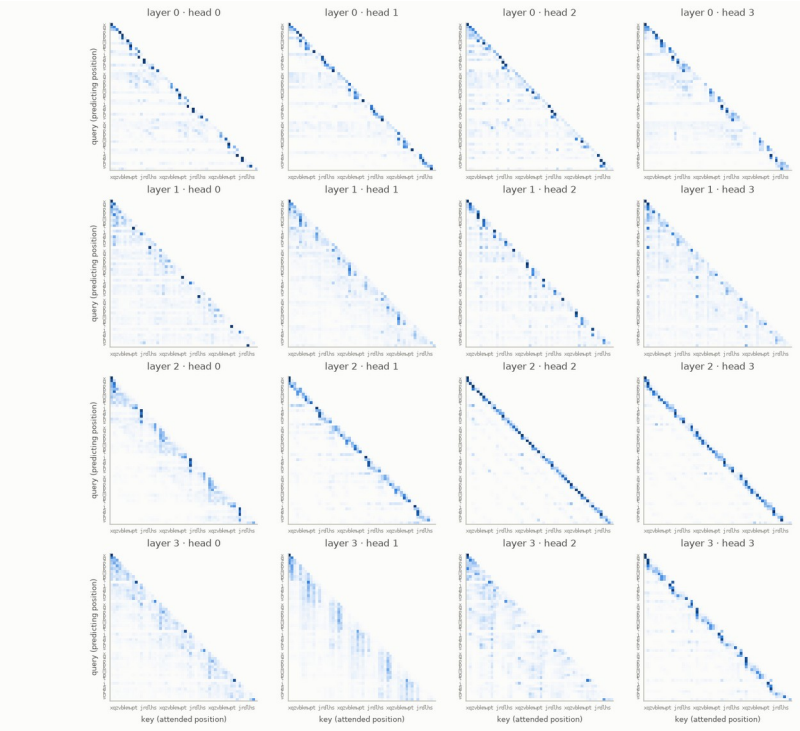


Figure D2. Repeated string `xqzvbkmwpt jrdlhs` × 3. L2·H2 stays a clean previous-token head. No head shows the offset diagonal an induction head would draw.

E. Extra credit: a GRU cell written gate by gate

`scripts/gru_scratch.py` implements PyTorch's GRU equations one gate at a time, in the style of `LSTMScratch`: r = σ(W_ir x + b_ir + W_hr h + b_hr), z = σ(W_iz x + b_iz + W_hz h + b_hz), n = tanh(W_in x + b_in + r ⊙ (W_hn h + b_hn)), h′ = (1 − z) ⊙ n + z ⊙ h. One detail matters for an exact match: the reset gate multiplies the *projected* hidden state including its bias, so unlike LSTMScratch the input and hidden projections must be kept as two separate linear layers. Weights are copied from `nn.GRU`, and outputs, final states and all weight gradients are compared.

Table E1. Maximum absolute difference vs. torch.nn.GRU (float64 unless noted).

case	outputs	final h	gradients	result
1 layer, zero h0	2.8e-16	1.7e-16	1.8e-14	PASS
1 layer, random h0	3.3e-16	1.3e-16	1.4e-14	PASS
2 layers, random h0	5.6e-16	2.2e-16	8.0e-15	PASS
3 layers, T = 128, H = 64	3.3e-16	1.7e-16	3.6e-14	PASS
2 layers, B = 1, T = 1	2.8e-17	5.6e-17	4.4e-16	PASS
float32, T = 50	1.3e-7	—	—	PASS

The differences are at the level of floating-point rounding (about 1e-16 in float64, 1e-7 in float32), so the gate-by-gate cell computes exactly what `nn.GRU` computes, gradients included.

F. Reproducing the results

From the `LLM Homework1` folder: `bash hw1/setup.sh` (environment, course repository, tests, benchmark), then `PAR_GPU=8 PAR_CPU=6 bash hw1/run_all.sh all`. The sweep files are `hw1/sweeps/hw1_char_models.yaml` (Part 2), `hw1_context.yaml` (Part 3, which reuses the seq_len = 256 runs of Part 2) and `hw1_untrained.yaml` (Part 1b). The analysis scripts are in `hw1/scripts/`: metrics_by_hand, untrained_vs_lnV, init_loss, plot_train_val, plot_context, attention_heads, gru_scratch and run_info. Outputs, including `summary.csv` and `summary.md` from analyze.py, are in `hw1/results/`. Finished runs are skipped on a rerun.

Sample from gpt_modern (seed 1337, temperature 0.8, top-k 50): *"CORIOLANUS: The England's life and thousand son, and with him, / Which are then should before the life: ... WARWICK: That purged wears of an oath shall be this?"* The model gets the speaker format and the line rhythm right, and writes mostly real words, but the sentences carry no meaning.